BANK LOAN ANALYSIS

In [1]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt #creating 2d graphs
import seaborn as sns #creating more statistical analysis
import warnings #if warnings will be there then it will give more structured war
import plotly.express as px #mostly used for interactive line,area & scatter cha
```

In [2]:
```
df = pd.read_excel('financial_loan.xlsx')
```

In [3]:
```
df.head()
```

Out[3]:

	id	address_state	application_type	emp_length	emp_title	grade	home_own
0	1077430	GA	INDIVIDUAL	< 1 year	Ryder	C	
1	1072053	CA	INDIVIDUAL	9 years	MKC Accounting	E	
2	1069243	CA	INDIVIDUAL	4 years	Chemat Technology Inc	C	
3	1041756	TX	INDIVIDUAL	< 1 year	barnes distribution	B	MOR
4	1068350	IL	INDIVIDUAL	10+ years	J&J Steel Inc	A	MOR

5 rows × 24 columns

In [4]:
```
df.tail()
```

Out[4]:

	id	address_state	application_type	emp_length	emp_title	grade	home_ov
38571	803452	NJ	INDIVIDUAL	< 1 year	Joseph M Sanzari Company	C	MC
38572	970377	NY	INDIVIDUAL	8 years	Swat Fame	C	
38573	875376	CA	INDIVIDUAL	5 years	Anaheim Regional Medical Center	D	
38574	972997	NY	INDIVIDUAL	5 years	Brooklyn Radiology	D	
38575	682952	NY	INDIVIDUAL	4 years	Allen Edmonds	F	

5 rows × 24 columns

In [5]:
```
df.shape
```

Out[5]:
```
(38576, 24)
```

In [6]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 38576 entries, 0 to 38575
Data columns (total 24 columns):
 #   Column                 Non-Null Count  Dtype
---  ------                 --------------  -----
 0   id                     38576 non-null  int64
 1   address_state          38576 non-null  object
 2   application_type        38576 non-null  object
 3   emp_length             38576 non-null  object
 4   emp_title              37138 non-null  object
 5   grade                  38576 non-null  object
 6   home_ownership         38576 non-null  object
 7   issue_date             38576 non-null  datetime64[ns]
 8   last_credit_pull_date  38576 non-null  datetime64[ns]
 9   last_payment_date      38576 non-null  datetime64[ns]
 10  loan_status            38576 non-null  object
 11  next_payment_date      38576 non-null  datetime64[ns]
 12  member_id              38576 non-null  int64
 13  purpose                38576 non-null  object
 14  sub_grade              38576 non-null  object
 15  term                   38576 non-null  object
 16  verification_status    38576 non-null  object
 17  annual_income          38576 non-null  float64
 18  dti                    38576 non-null  float64
 19  installment            38576 non-null  float64
 20  int_rate               38576 non-null  float64
 21  loan_amount            38576 non-null  int64
 22  total_acc              38576 non-null  int64
 23  total_payment          38576 non-null  int64
dtypes: datetime64[ns](4), float64(4), int64(5), object(11)
memory usage: 7.1+ MB
```

In [7]:
```
df.dtypes
```

Out[7]:
```
id                                int64
address_state                    object
application_type                 object
emp_length                       object
emp_title                        object
grade                            object
home_ownership                   object
issue_date               datetime64[ns]
last_credit_pull_date    datetime64[ns]
last_payment_date        datetime64[ns]
loan_status                      object
next_payment_date        datetime64[ns]
member_id                         int64
purpose                          object
sub_grade                        object
term                             object
verification_status              object
annual_income                   float64
dti                             float64
installment                     float64
int_rate                        float64
loan_amount                       int64
total_acc                         int64
total_payment                     int64
dtype: object
```

In [8]:
```
df.describe()
```

Out[8]:

	id	issue_date	last_credit_pull_date	last_payment_date	next_p
count	3.857600e+04	38576	38576	38576	
mean	6.810371e+05	2021-07-16 02:31:35.562007040	2021-06-08 13:36:34.193280512	2021-06-26 09:52:08.909166080	20:42:
min	5.473400e+04	2021-01-01 00:00:00	2021-01-08 00:00:00	2021-01-08 00:00:00	2021-(
25%	5.135170e+05	2021-04-11 00:00:00	2021-04-15 00:00:00	2021-03-16 00:00:00	2021-(
50%	6.627280e+05	2021-07-11 00:00:00	2021-05-16 00:00:00	2021-06-14 00:00:00	2021-(
75%	8.365060e+05	2021-10-11 00:00:00	2021-08-13 00:00:00	2021-09-15 00:00:00	2021-1
max	1.077501e+06	2021-12-12 00:00:00	2022-01-20 00:00:00	2021-12-15 00:00:00	2022-(
std	2.113246e+05	NaN	NaN	NaN	

Key performance Indicator

In [9]:
```
#Total loan Applications

Total_loan_application = df['id'].count()
print("Total loan Application:", Total_loan_application)
```

```
Total loan Application: 38576
```

In [10]:
```
# MTD loan applications

latest_issue_date = df['issue_date'].max()  #to get the latest date by max func
latest_year = latest_issue_date.year
latest_month = latest_issue_date.month

mtd_data = df[(df['issue_date'].dt.year == latest_year) & (df['issue_date'].dt.m

mtd_loan_application = mtd_data['id'].count()

print(f"MTD Loan Application (for {latest_issue_date.strftime('%B %Y')}) : {mtd_
```

```
MTD Loan Application (for December 2021) : 4314
```

In [11]:
```
# Total funded amount

total_funded_amount = df['loan_amount'].sum()
total_funded_amount_millions = total_funded_amount/1000000
print(f"Total Funded Amount : ${total_funded_amount_millions:.2f}M")
```

```
Total Funded Amount : $435.76M
```

In [12]:
```
# MTS funded amount
```

```
latest_issue_date = df['issue_date'].max()  #to get the latest date by max func
latest_year = latest_issue_date.year
latest_month = latest_issue_date.month

mtd_data = df[(df['issue_date'].dt.year == latest_year) & (df['issue_date'].dt.m

mtd_total_funded_amount = mtd_data['loan_amount'].sum()

mtd_total_funded_amount_millions = mtd_total_funded_amount/1000000

print(f"MTD Funded Amount (for {latest_issue_date.strftime('%B %Y')}) : ${mtd_to
```

```
MTD Funded Amount (for December 2021) : $53.98M
```

In [13]:

```
#Total Amount Received

total_amount_received = df['total_payment'].sum()
total_amount_received_millions = total_amount_received/1000000
print(f"Total Amount Received: ${total_amount_received_millions:.2f}M")
```

```
Total Amount Received: $473.07M
```

In [14]:

```
# MTS amount received

latest_issue_date = df['issue_date'].max()  #to get the latest date by max func
latest_year = latest_issue_date.year
latest_month = latest_issue_date.month

mtd_data = df[(df['issue_date'].dt.year == latest_year) & (df['issue_date'].dt.m

mtd_total_amount_received = mtd_data['total_payment'].sum()

mtd_total_amount_received_millions = mtd_total_amount_received/1000000

print(f"MTD Amount Received (for {latest_issue_date.strftime('%B %Y')}) : ${mtd_
```

```
MTD Amount Received (for December 2021) : $58.07M
```

In [15]:

```
# Average Interest rate

average_interest_rate = df['int_rate'].mean()*100
print(f"Average Int Rate: {average_interest_rate:.2f}%")
```

```
Average Int Rate: 12.05%
```

In [16]:

```
# Average Debt-to-Income Ratio (DTI)

average_Debt_to_income_Ratio = df['dti'].mean()*100
print(f"Average_Debt_to_Income_Ratio: {average_Debt_to_income_Ratio:.2f}%")
```

```
Average_Debt_to_Income_Ratio: 13.33%
```

Good Loan Metrics

In [17]:

```
good_loan = df[df['loan_status'].isin(['Fully Paid','Current'])]

total_loan_applications = df['id'].count()

good_loan_applications = good_loan['id'].count()
good_loan_funded_amount = good_loan['loan_amount'].sum()
good_loan_amount_received = good_loan['total_payment'].sum()
```

```
good_loan_funded_amount_millions = good_loan_funded_amount/1000000
good_loan_amount_received_millions = good_loan_amount_received/1000000

good_loan_percentage = (good_loan_applications/total_loan_applications) * 100

print(f"Good Loan Applications: {good_loan_applications}")
print(f"Good Loan Funded Amount: ${good_loan_funded_amount_millions:.2f}M")
print(f"Good Loan Amount received: ${good_loan_amount_received_millions:.2f}M")
print(f"Good Loan Percentage: {good_loan_percentage:.2f}%")
```

```
Good Loan Applications: 33243
Good Loan Funded Amount: $370.22M
Good Loan Amount received: $435.79M
Good Loan Percentage: 86.18%
```

Bad Loan Metrics

In [18]:
```
bad_loan = df[df['loan_status'].isin(['Charged Off'])]

total_loan_applications = df['id'].count()

bad_loan_applications = bad_loan['id'].count()
bad_loan_funded_amount = bad_loan['loan_amount'].sum()
bad_loan_amount_received = bad_loan['total_payment'].sum()

bad_loan_funded_amount_millions = bad_loan_funded_amount/1000000
bad_loan_amount_received_millions = bad_loan_amount_received/1000000

bad_loan_percentage = (bad_loan_applications/total_loan_applications) * 100

print(f"Bad Loan Applications: {bad_loan_applications}")
print(f"Bad Loan Funded Amount: ${bad_loan_funded_amount_millions:.2f}M")
print(f"Bad Loan Amount received: ${bad_loan_amount_received_millions:.2f}M")
print(f"Bad Loan Percentage: {bad_loan_percentage:.2f}%")
```

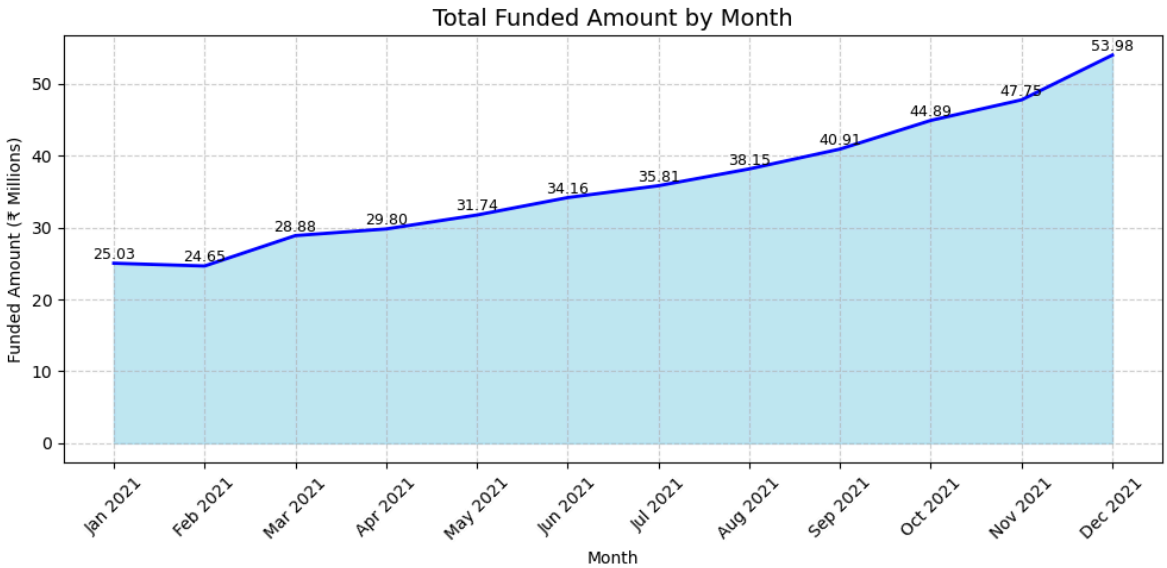
```
Bad Loan Applications: 5333
Bad Loan Funded Amount: $65.53M
Bad Loan Amount received: $37.28M
Bad Loan Percentage: 13.82%
```

Visualisations

monthly trends by issue date for total funded amount

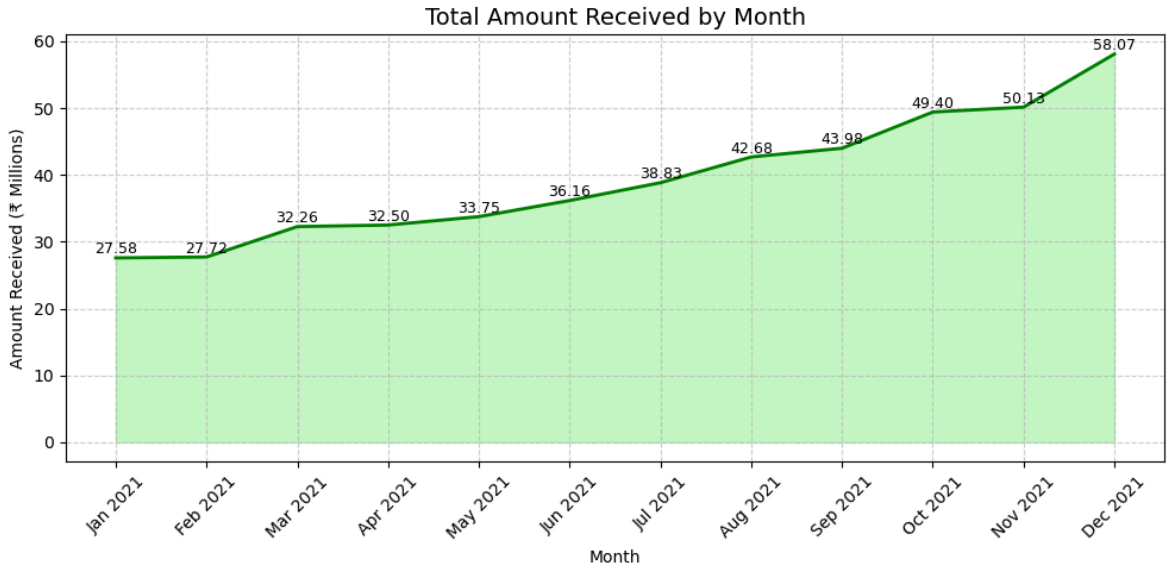
In [19]:
```
monthly_funded = (
    df.sort_values('issue_date')
      .assign(month_name=lambda x: x['issue_date'].dt.strftime('%b %Y'))
      .groupby('month_name',sort=False)['loan_amount']
      .sum()
      .div(1_000_000)
      .reset_index(name='loan_amount_millions')
)

plt.figure(figsize=(10,5))
plt.fill_between(monthly_funded['month_name'],monthly_funded['loan_amount_millio
plt.plot(monthly_funded['month_name'],monthly_funded['loan_amount_millions'],col
```

```
for i,row in monthly_funded.iterrows():
    plt.text(i,row['loan_amount_millions'] + 0.1, f"{row['loan_amount_millions']
             ha = 'center', va='bottom',fontsize=9, rotation=0, color='black')

plt.title('Total Funded Amount by Month', fontsize=14)
plt.xlabel('Month')
plt.ylabel('Funded Amount (₹ Millions)')
plt.xticks(ticks=range(len(monthly_funded)), labels=monthly_funded['month_name']
plt.grid(True, linestyle='--', alpha=0.6)
plt.tight_layout()
plt.show()
```



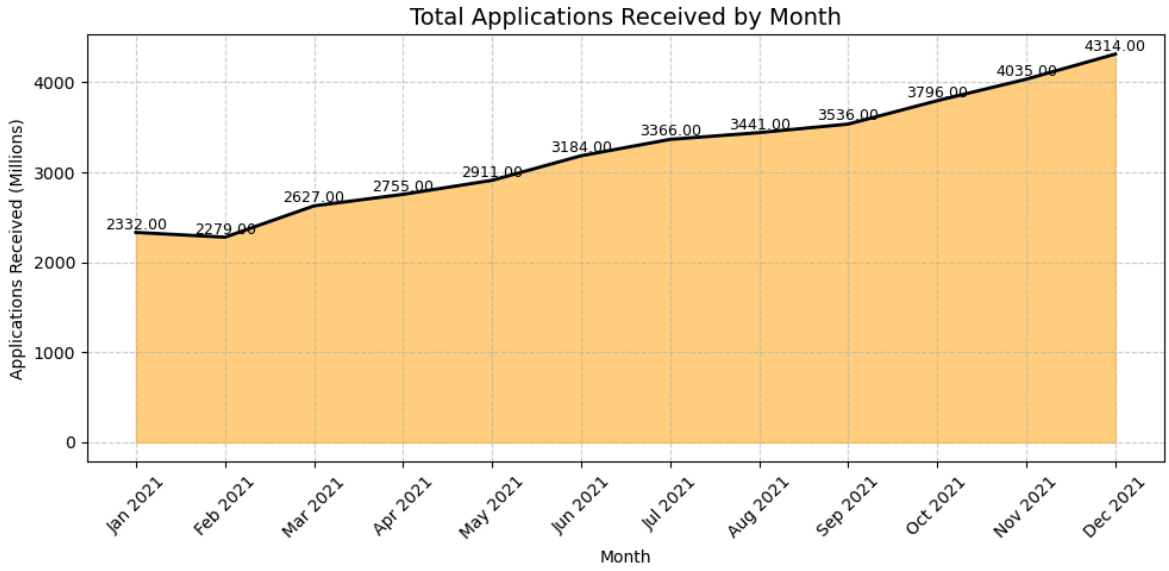
monthly trends by issue date for total amount received

In [20]:
```
monthly_received = (
    df.sort_values('issue_date')
      .assign(month_name=lambda x: x['issue_date'].dt.strftime('%b %Y'))
      .groupby('month_name',sort=False)['total_payment']
      .sum()
      .div(1000000)
      .reset_index(name='received_amount_millions')
)

plt.figure(figsize=(10,5))
plt.fill_between(monthly_received['month_name'],monthly_received['received_amoun
plt.plot(monthly_received['month_name'],monthly_received['received_amount_millio

for i,row in monthly_received.iterrows():
    plt.text(i,row['received_amount_millions'] + 0.1, f"{row['received_amount_mi
             ha = 'center', va='bottom',fontsize=9, rotation=0, color='black')

plt.title('Total Amount Received by Month', fontsize=14)
plt.xlabel('Month')
plt.ylabel('Amount Received (₹ Millions)')
plt.xticks(ticks=range(len(monthly_received)), labels=monthly_received['month_na
plt.grid(True, linestyle='--', alpha=0.6)
plt.tight_layout()
plt.show()
```



monthly trends by issue date for total loan applications

In [21]:
```
monthly_applications = (
    df.sort_values('issue_date')
      .assign(month_name=lambda x: x['issue_date'].dt.strftime('%b %Y'))
      .groupby('month_name',sort=False)['id']
      .count()
      .reset_index(name='loan_applications_count')
)

plt.figure(figsize=(10,5))
plt.fill_between(monthly_applications['month_name'],monthly_applications['loan_a
plt.plot(monthly_applications['month_name'],monthly_applications['loan_applicati

for i,row in monthly_applications.iterrows():
    plt.text(i,row['loan_applications_count'] + 0.1, f"{row['loan_applications_c
             ha = 'center', va='bottom',fontsize=9, rotation=0, color='black')

plt.title('Total Applications Received by Month', fontsize=14)
plt.xlabel('Month')
plt.ylabel('Applications Received (Millions)')
plt.xticks(ticks=range(len(monthly_applications)), labels=monthly_applications['
plt.grid(True, linestyle='--', alpha=0.6)
plt.tight_layout()
plt.show()
```



Regional Analysis by State for Total Funded Amount

In [22]:
```
df.head()
```

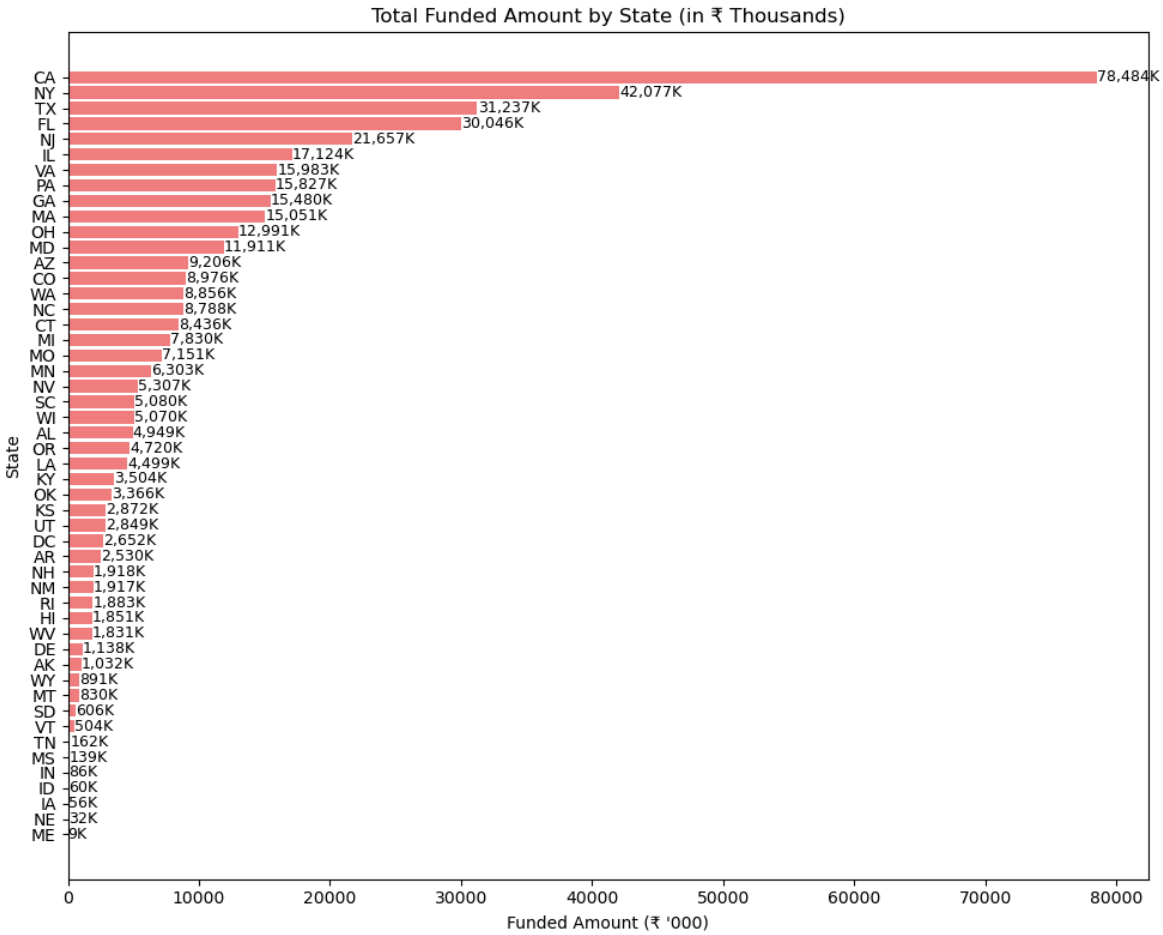
Out[22]:

	id	address_state	application_type	emp_length	emp_title	grade	home_own
0	1077430	GA	INDIVIDUAL	< 1 year	Ryder	C	
1	1072053	CA	INDIVIDUAL	9 years	MKC Accounting	E	
2	1069243	CA	INDIVIDUAL	4 years	Chemat Technology Inc	C	
3	1041756	TX	INDIVIDUAL	< 1 year	barnes distribution	B	MOR
4	1068350	IL	INDIVIDUAL	10+ years	J&J Steel Inc	A	MOR

5 rows × 24 columns

In [26]:
```
state_funding = df.groupby('address_state')['loan_amount'].sum().sort_values(asc
state_funding_thousands = state_funding / 1000

plt.figure(figsize=(10,8))
bars = plt.barh(state_funding_thousands.index, state_funding_thousands.values, c

for bar in bars:
    width = bar.get_width()
    plt.text(width + 10, bar.get_y() + bar.get_height() / 2,
        f'{width:,.0f}K', va='center', fontsize=9)

plt.title('Total Funded Amount by State (in ₹ Thousands)')
plt.xlabel('Funded Amount (₹ \'000)')
plt.ylabel('State')
```

```
plt.tight_layout()
plt.show()
```



Regional Analysis by State for Total Received Amount

In [28]:
```
df.head()
```

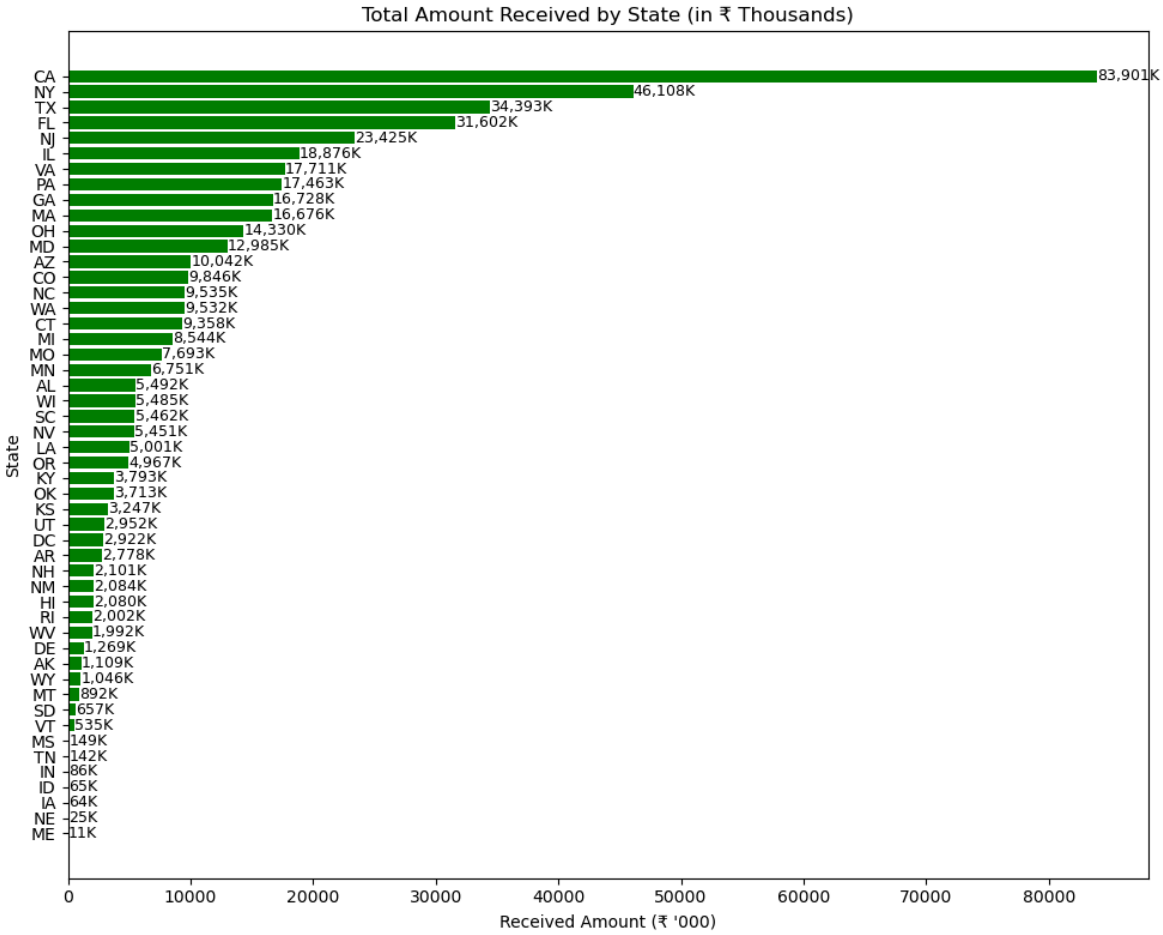
Out[28]:

	id	address_state	application_type	emp_length	emp_title	grade	home_own
0	1077430	GA	INDIVIDUAL	< 1 year	Ryder	C	
1	1072053	CA	INDIVIDUAL	9 years	MKC Accounting	E	
2	1069243	CA	INDIVIDUAL	4 years	Chemat Technology Inc	C	
3	1041756	TX	INDIVIDUAL	< 1 year	barnes distribution	B	MOR
4	1068350	IL	INDIVIDUAL	10+ years	J&J Steel Inc	A	MOR

5 rows × 24 columns

In [29]:
```
state_funding = df.groupby('address_state')['total_payment'].sum().sort_values(a
state_funding_thousands = state_funding / 1000
```

```
plt.figure(figsize=(10,8))
bars = plt.barh(state_funding_thousands.index, state_funding_thousands.values, c

for bar in bars:
    width = bar.get_width()
    plt.text(width + 10, bar.get_y() + bar.get_height() / 2,
        f'{width:,.0f}K', va='center', fontsize=9)

plt.title('Total Amount Received by State (in ₹ Thousands)')
plt.xlabel('Received Amount (₹ \'000)')
plt.ylabel('State')
plt.tight_layout()
plt.show()
```



Regional Analysis by State for Total Applications Received

In [30]:
```
df.head()
```

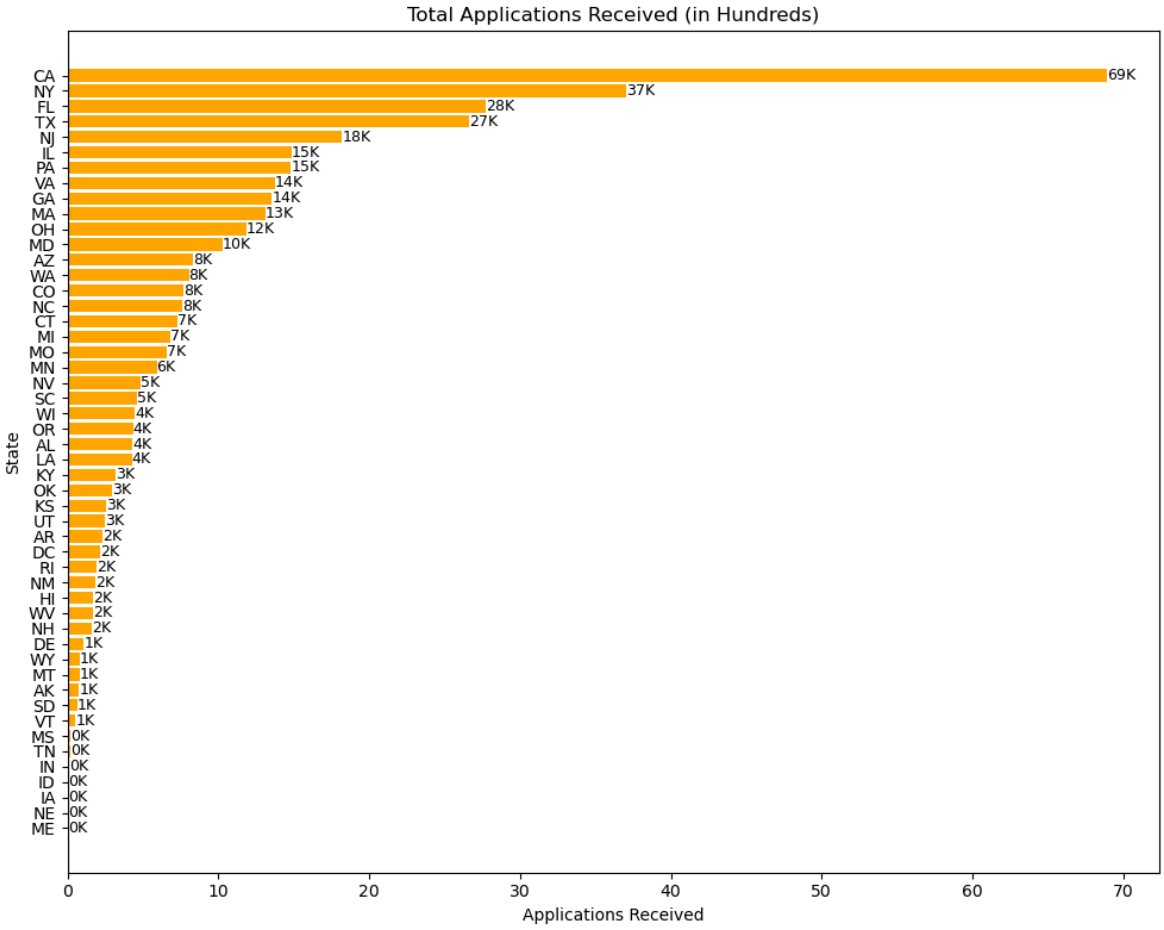
Out[30]:

	id	address_state	application_type	emp_length	emp_title	grade	home_own
0	1077430	GA	INDIVIDUAL	< 1 year	Ryder	C	
1	1072053	CA	INDIVIDUAL	9 years	MKC Accounting	E	
2	1069243	CA	INDIVIDUAL	4 years	Chemat Technology Inc	C	
3	1041756	TX	INDIVIDUAL	< 1 year	barnes distribution	B	MOR
4	1068350	IL	INDIVIDUAL	10+ years	J&J Steel Inc	A	MOR

5 rows × 24 columns

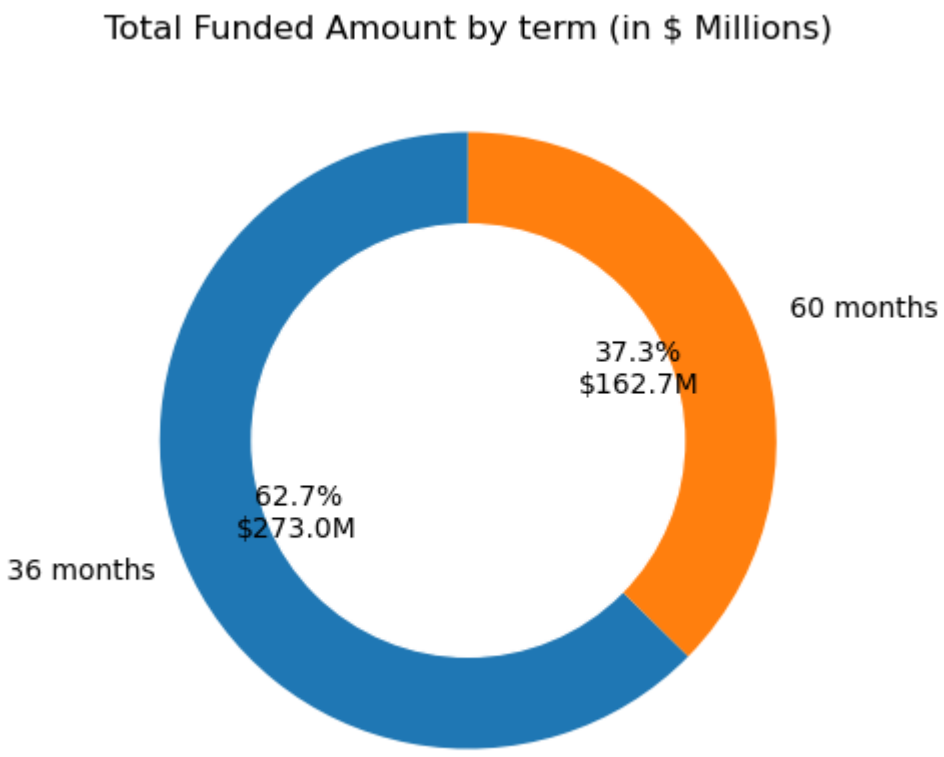
In [37]:

```
state_applications = df.groupby('address_state')['id'].count().sort_values(ascen
state_applications_hundreds = state_applications / 100

plt.figure(figsize=(10,8))
bars = plt.barh(state_applications_hundreds.index, state_applications_hundreds.v

max_val = state_applications_hundreds.max()  # ← get max value
offset = max_val * 0.01

for bar in bars:
    width = bar.get_width()
    plt.text(width, bar.get_y() + bar.get_height() / 2,
        f'{width:,.0f}K',va='center',fontsize=9)

plt.title('Total Applications Received (in Hundreds)')
plt.xlabel('Applications Received')
plt.ylabel('State')
plt.tight_layout()
plt.show()
```



Long Term Analysis by Total Funded Amount

In [39]:
```
term_funding_millions = df.groupby('term')['loan_amount'].sum() / 1000000

plt.figure(figsize=(5,5))
plt.pie(
    term_funding_millions,
    labels = term_funding_millions.index,
    autopct=lambda p: f"{p:.1f}%\n${p*sum(term_funding_millions)/100:.1f}M",
    startangle = 90,
    wedgeprops={'width': 0.4}
)
plt.gca().add_artist(plt.Circle((0,0), 0.70, color='white'))
plt.title('Total Funded Amount by term (in $ Millions)')
plt.show()
```



Long Term Analysis by Total Amount Received

In [40]:
```
df.head()
```

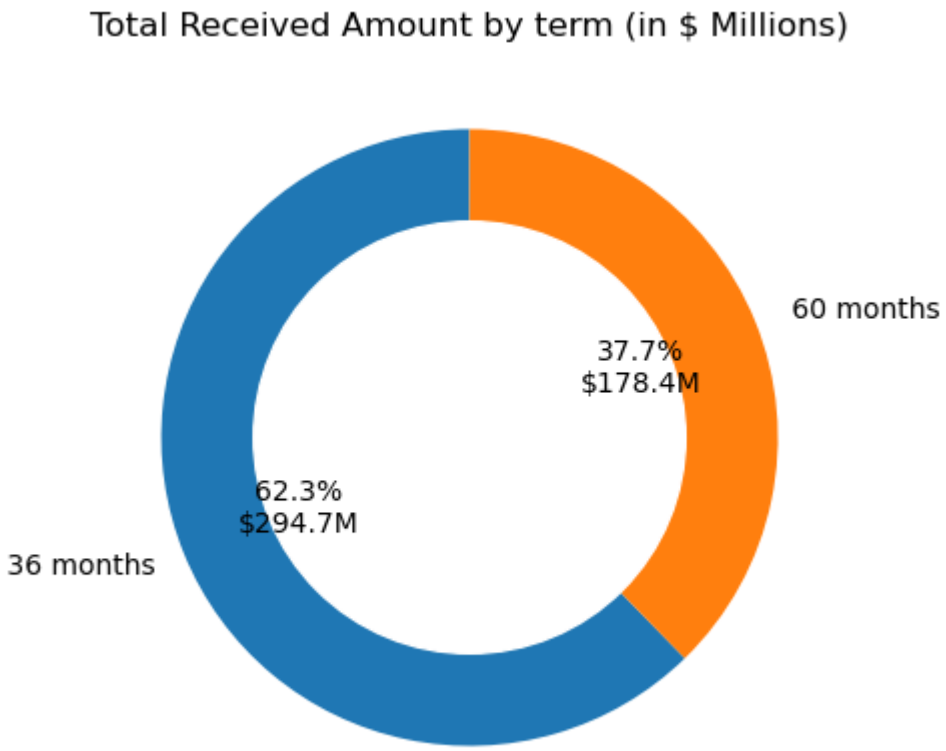
Out[40]:

	id	address_state	application_type	emp_length	emp_title	grade	home_own
0	1077430	GA	INDIVIDUAL	< 1 year	Ryder	C	
1	1072053	CA	INDIVIDUAL	9 years	MKC Accounting	E	
2	1069243	CA	INDIVIDUAL	4 years	Chemat Technology Inc	C	
3	1041756	TX	INDIVIDUAL	< 1 year	barnes distribution	B	MOR
4	1068350	IL	INDIVIDUAL	10+ years	J&J Steel Inc	A	MOR

5 rows × 24 columns

In [42]:
```
term_received_millions = df.groupby('term')['total_payment'].sum() / 1000000

plt.figure(figsize=(5,5))
plt.pie(
    term_received_millions,
    labels = term_received_millions.index,
    autopct=lambda p: f"{p:.1f}%\n${p*sum(term_received_millions)/100:.1f}M",
    startangle = 90,
```

```
    wedgeprops={'width': 0.4}
)
plt.gca().add_artist(plt.Circle((0,0), 0.70, color='white'))
plt.title('Total Received Amount by term (in $ Millions)')
plt.show()
```



Long Term Analysis by Total Applications Received

In [43]:
```
df.head()
```

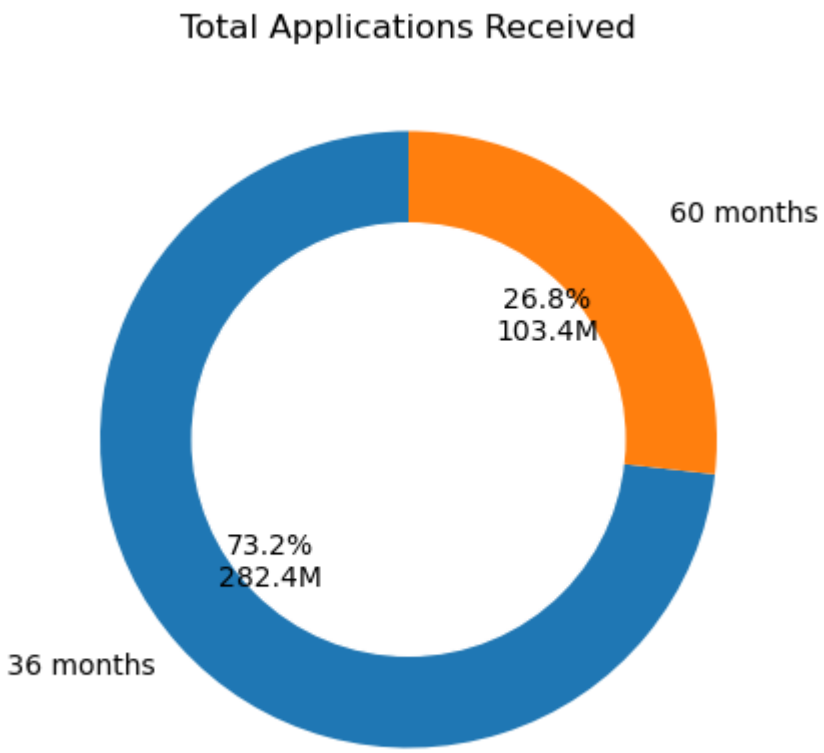
Out[43]:

	id	address_state	application_type	emp_length	emp_title	grade	home_own
0	1077430	GA	INDIVIDUAL	< 1 year	Ryder	C	
1	1072053	CA	INDIVIDUAL	9 years	MKC Accounting	E	
2	1069243	CA	INDIVIDUAL	4 years	Chemat Technology Inc	C	
3	1041756	TX	INDIVIDUAL	< 1 year	barnes distribution	B	MOR
4	1068350	IL	INDIVIDUAL	10+ years	J&J Steel Inc	A	MOR

5 rows × 24 columns

In [46]:
```
term_applications_received = df.groupby('term')['id'].count()/100
```

```
plt.figure(figsize=(5,5))
plt.pie(
    term_applications_received,
    labels = term_applications_received.index,
    autopct=lambda p: f"{p:.1f}%\n{p*sum(term_applications_received)/100:.1f}M",
    startangle = 90,
    wedgeprops={'width': 0.4}
)
plt.gca().add_artist(plt.Circle((0,0), 0.70, color='white'))
plt.title('Total Applications Received')
plt.show()
```



Employee Length Analysis by Total Funded Amount

In [48]:
```
df.head()
```

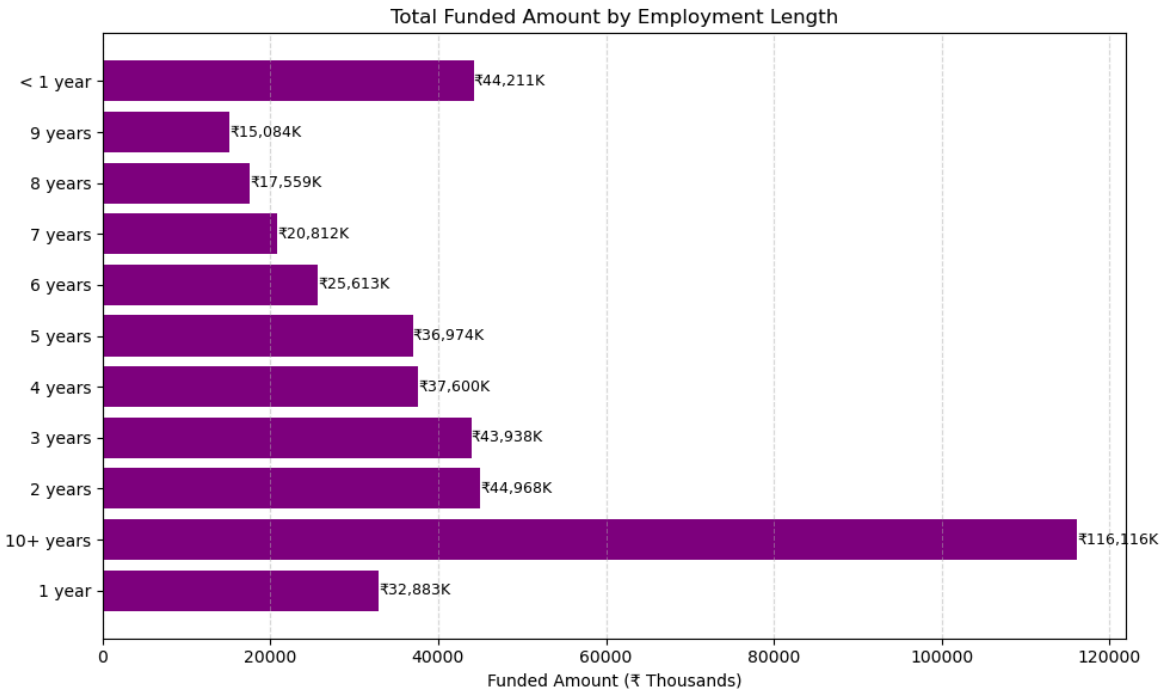
Out[48]:

	id	address_state	application_type	emp_length	emp_title	grade	home_own
0	1077430	GA	INDIVIDUAL	< 1 year	Ryder	C	
1	1072053	CA	INDIVIDUAL	9 years	MKC Accounting	E	
2	1069243	CA	INDIVIDUAL	4 years	Chemat Technology Inc	C	
3	1041756	TX	INDIVIDUAL	< 1 year	barnes distribution	B	MOR
4	1068350	IL	INDIVIDUAL	10+ years	J&J Steel Inc	A	MOR

5 rows × 24 columns

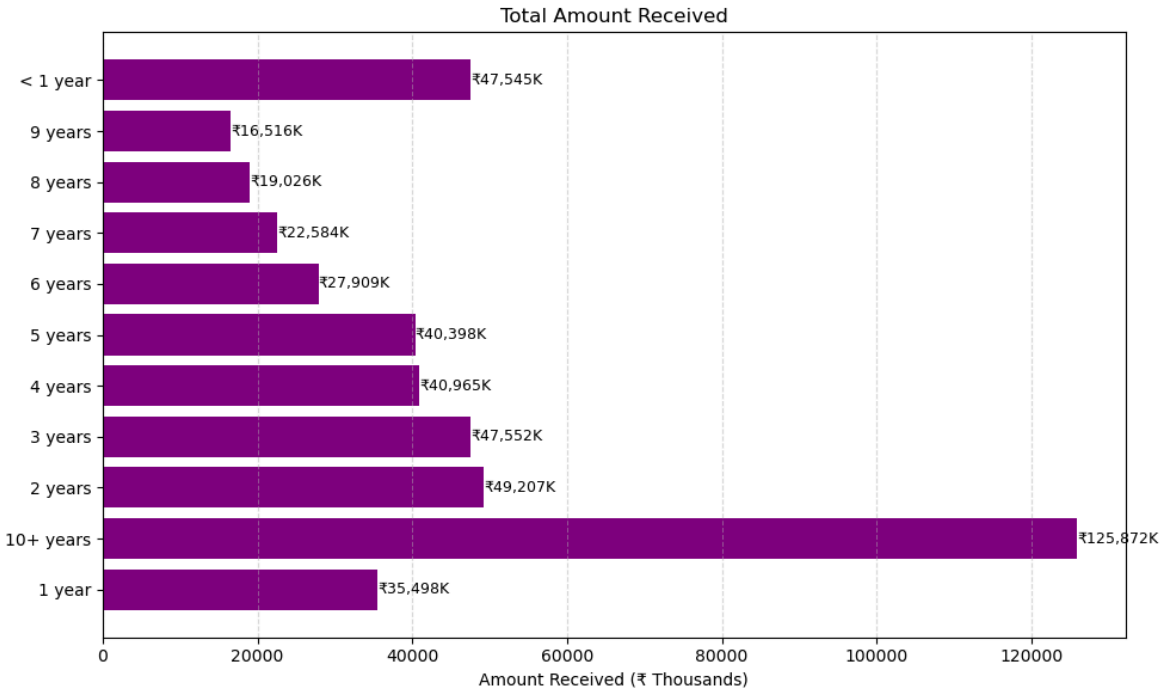
In [50]:

```
emp_funding_thousands = df.groupby('emp_length')['loan_amount'].sum() / 1000

plt.figure(figsize=(10, 6))
bars = plt.barh(emp_funding_thousands.index, emp_funding_thousands, color='purpl

for bar in bars:
    width = bar.get_width()
    plt.text(width + 5, bar.get_y() + bar.get_height() / 2,
             f"₹{width:,.0f}K", va='center', fontsize=9)

plt.xlabel("Funded Amount (₹ Thousands)")
plt.title("Total Funded Amount by Employment Length")
plt.grid(axis='x', linestyle='--', alpha=0.5)
plt.tight_layout()
plt.show()
```



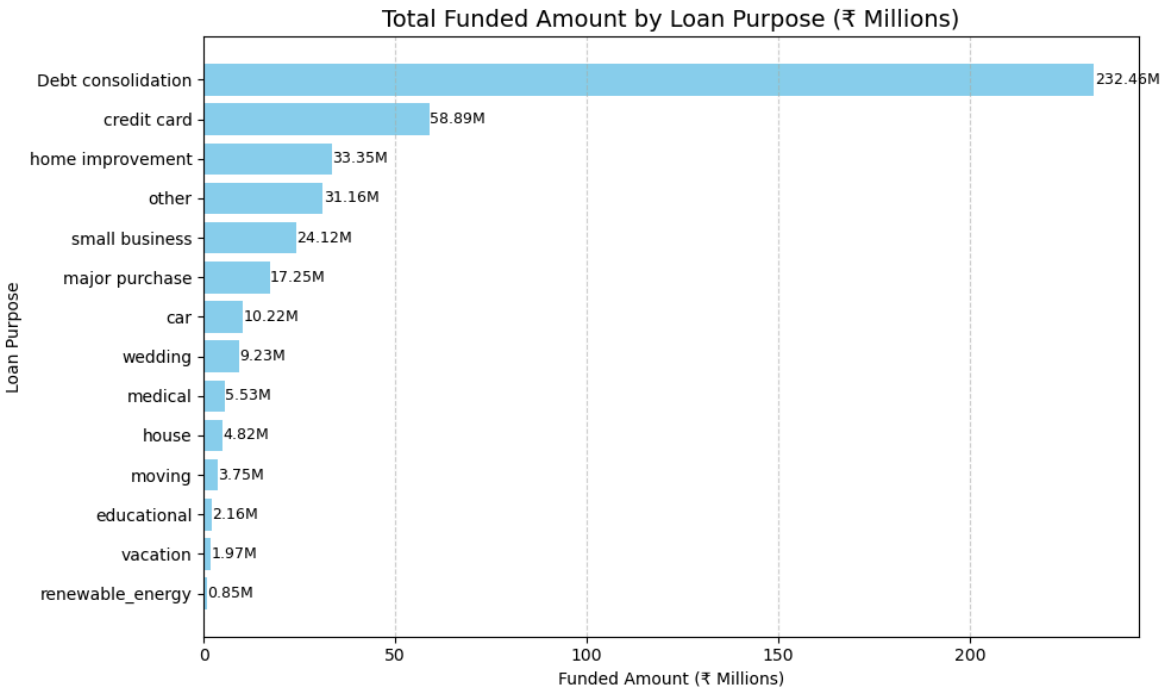
Employee Length Analysis by Total Amount Received

In [51]:
```
emp_amount_received_thousands = df.groupby('emp_length')['total_payment'].sum()

plt.figure(figsize=(10, 6))
bars = plt.barh(emp_amount_received_thousands.index, emp_amount_received_thousan

for bar in bars:
    width = bar.get_width()
    plt.text(width + 5, bar.get_y() + bar.get_height() / 2,
             f"₹{width:,.0f}K", va='center', fontsize=9)

plt.xlabel("Amount Received (₹ Thousands)")
plt.title("Total Amount Received")
plt.grid(axis='x', linestyle='--', alpha=0.5)
plt.tight_layout()
plt.show()
```



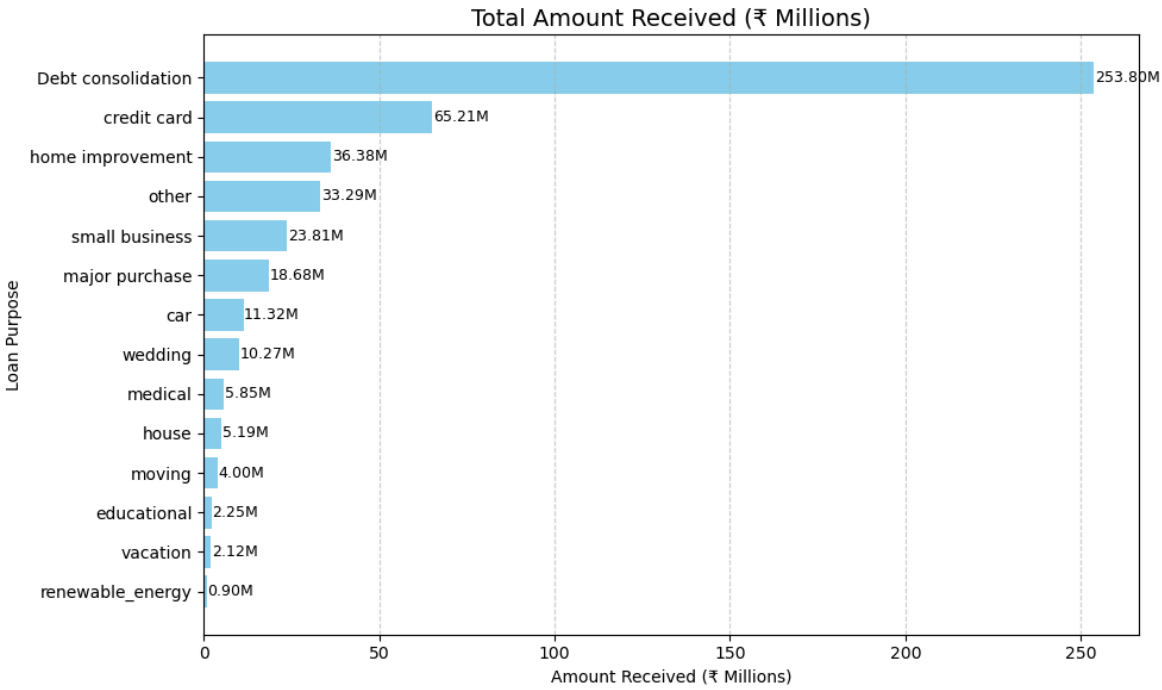
Loan Purpose by Total Funded Amount

In [52]:
```
purpose_funding_millions = (df.groupby('purpose')['loan_amount'].sum().sort_valu

plt.figure(figsize=(10, 6))
bars = plt.barh(purpose_funding_millions.index, purpose_funding_millions.values,

for bar in bars:
    width = bar.get_width()
    plt.text(width + 0.1, bar.get_y() + bar.get_height()/2,
             f'{width:.2f}M', va='center', fontsize=9)

plt.title('Total Funded Amount by Loan Purpose (₹ Millions)', fontsize=14)
plt.xlabel('Funded Amount (₹ Millions)')
plt.ylabel('Loan Purpose')
plt.grid(axis='x', linestyle='--', alpha=0.6)
plt.tight_layout()
plt.show()
```



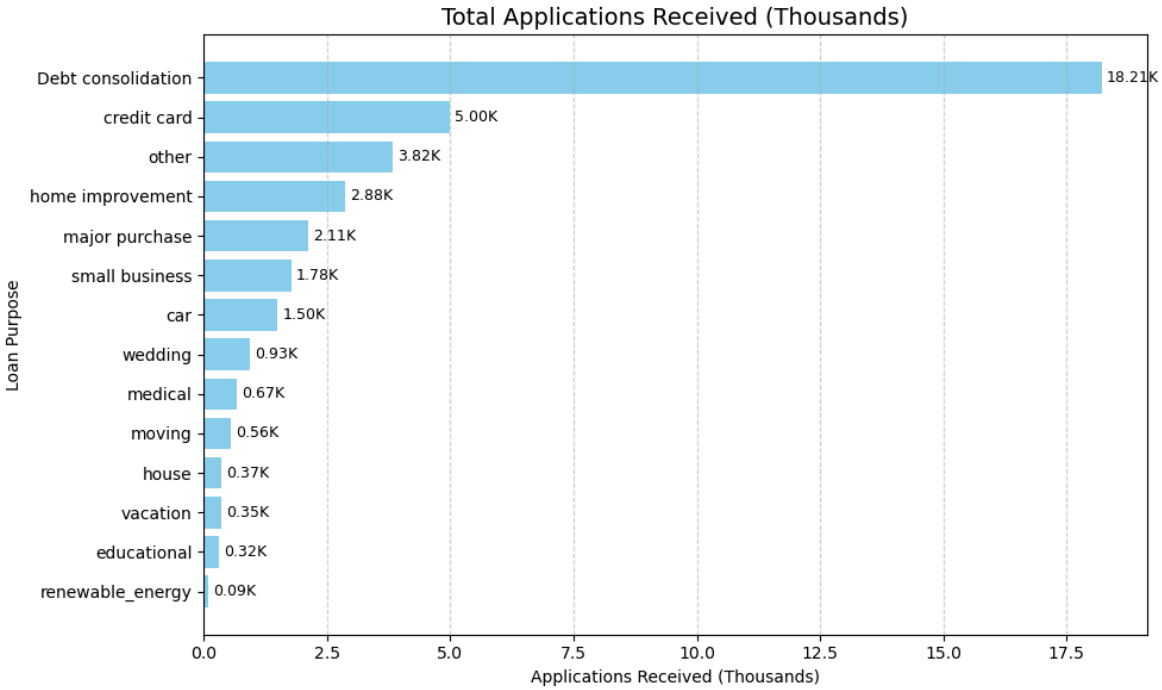
Loan Purpose by Total Amount Received

In [54]:
```
purpose_amount_received_millions = (df.groupby('purpose')['total_payment'].sum()

plt.figure(figsize=(10, 6))
bars = plt.barh(purpose_amount_received_millions.index, purpose_amount_received_

for bar in bars:
    width = bar.get_width()
    plt.text(width + 0.1, bar.get_y() + bar.get_height()/2,
             f'{width:.2f}M', va='center', fontsize=9)

plt.title('Total Amount Received (₹ Millions)', fontsize=14)
plt.xlabel('Amount Received (₹ Millions)')
plt.ylabel('Loan Purpose')
plt.grid(axis='x', linestyle='--', alpha=0.6)
plt.tight_layout()
plt.show()
```



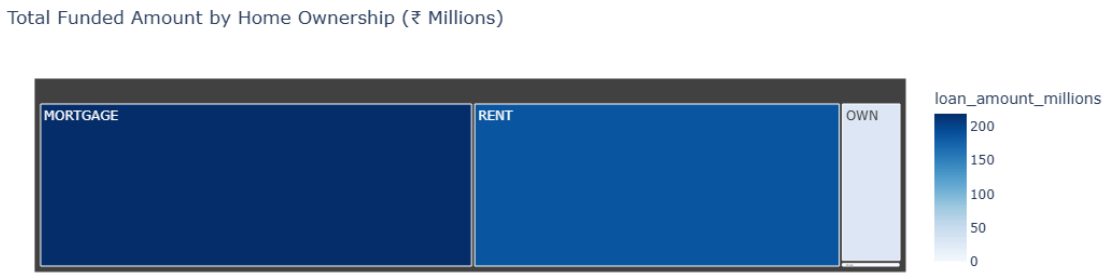
Loan Purpose by Total Applications Received

In [57]:
```
purpose_applications_received_thousands = (df.groupby('purpose')['id'].count().s

plt.figure(figsize=(10, 6))
bars = plt.barh(purpose_applications_received_thousands.index, purpose_applicati

for bar in bars:
    width = bar.get_width()
    plt.text(width + 0.1, bar.get_y() + bar.get_height()/2,
             f'{width:.2f}K', va='center', fontsize=9)

plt.title('Total Applications Received (Thousands)', fontsize=14)
plt.xlabel('Applications Received (Thousands)')
plt.ylabel('Loan Purpose')
plt.grid(axis='x', linestyle='--', alpha=0.6)
plt.tight_layout()
plt.show()
```



Home Ownership by Total Funded Amount

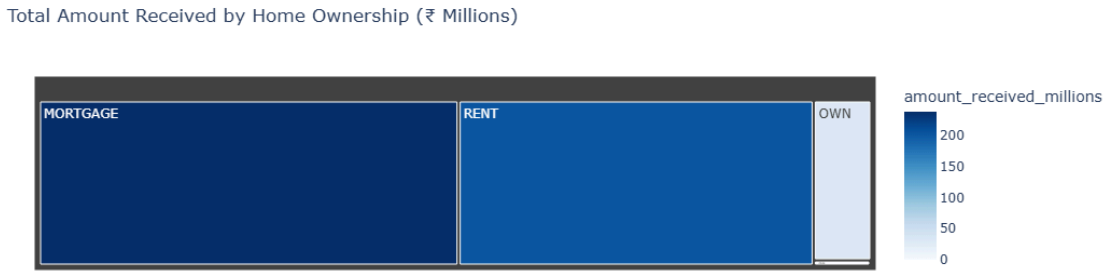
In [59]:
```
home_funding = df.groupby('home_ownership')['loan_amount'].sum().reset_index()
home_funding['loan_amount_millions'] = home_funding['loan_amount'] / 1000000

fig = px.treemap(
    home_funding,
    path=['home_ownership'],
    values='loan_amount_millions',
    color='loan_amount_millions',
    color_continuous_scale='Blues',
    title='Total Funded Amount by Home Ownership (₹ Millions)'
)

fig.show()
```



Home Ownership by Total Amount Received

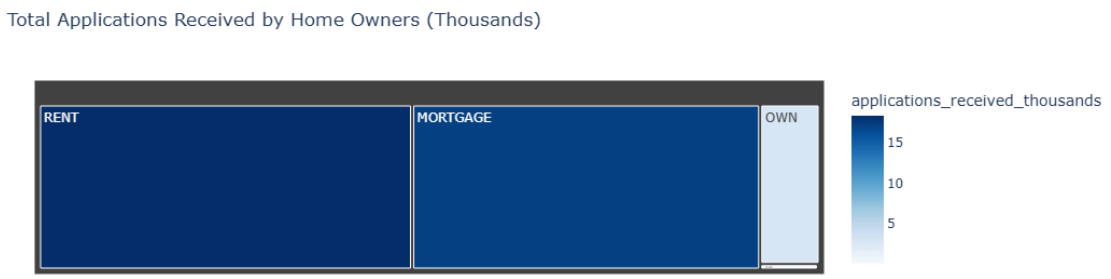
In [65]:
```
home_amount_received = df.groupby('home_ownership')['total_payment'].sum().reset
home_amount_received['amount_received_millions'] = home_amount_received['total_p

fig = px.treemap(
    home_amount_received,
    path=['home_ownership'],
```

```
    values='amount_received_millions',
    color='amount_received_millions',
    color_continuous_scale='Blues',
    title='Total Amount Received by Home Ownership (₹ Millions)'
)

fig.show()
```



Home Ownership by Total Applications Received

In [67]:
```
home_applications_received = df.groupby('home_ownership')['id'].count().reset_in
home_applications_received['applications_received_thousands'] = home_application

fig = px.treemap(
    home_applications_received,
    path=['home_ownership'],
    values='applications_received_thousands',
    color='applications_received_thousands',
    color_continuous_scale='Blues',
    title='Total Applications Received by Home Owners (Thousands)'
)

fig.show()
```



Thank You

In [ ]: